React + TypeScript

A Comprehensive Guide to Building Type-Safe React Applications

Contents

1 What is TypeScript & Why Use It? 2
- 1.1 Understanding TypeScript . . . 2
- 1.2 Why TypeScript? . . . 2
- 1.3 TypeScript vs JavaScript . . . 2

2 Installing & Using TypeScript 4
- 2.1 Setting Up TypeScript . . . 4
- 2.2 Creating a TypeScript Configuration . . . 4
- 2.3 Compiling TypeScript . . . 4
- 2.4 Using TypeScript in React . . . 4

3 Exploring the Base Types 5
- 3.1 The Primitive Types . . . 5
- 3.2 Type Inference . . . 5

4 Working with Array & Object Types 6
- 4.1 Array Types . . . 6
- 4.2 Object Types . . . 6

5 Understanding Type Inference 8
- 5.1 How Type Inference Works . . . 8
- 5.2 When Inference Isn't Enough . . . 8

6 Using Union Types 9
- 6.1 Combining Multiple Types . . . 9
- 6.2 Type Narrowing . . . 9
- 6.3 Real-World Union Example . . . 9

7 Understanding Type Aliases 11
- 7.1 Creating Reusable Types . . . 11
- 7.2 Interfaces vs Type Aliases . . . 11
- 7.3 Type Aliases with Unions . . . 12

8 Functions & Function Types 13
- 8.1 Basic Function Types . . . 13
- 8.2 Function Type Expressions . . . 13
- 8.3 Callback Functions . . . 14

9 Diving Into Generics **15**
9.1 What Are Generics? . . . 15
9.2 Generic Functions . . . 15

10 A Closer Look At Generics **17**
10.1 Generic Interfaces . . . 17
10.2 Generic Constraints . . . 17
10.3 Default Type Parameters . . . 18

11 Creating a React + TypeScript Project **19**
11.1 Setup with Create React App . . . 19
11.2 Setup with Vite . . . 19
11.3 Project Structure . . . 19
11.4 First TypeScript React File . . . 19

12 Working with Components & TypeScript **21**
12.1 Functional Components . . . 21
12.2 Class Components . . . 21

13 Working with Props & TypeScript **23**
13.1 Typing Props . . . 23
13.2 Common Prop Types . . . 23
13.3 Spread Props . . . 24

14 Adding a Data Model **25**
14.1 Creating Type Definitions . . . 25
14.2 Working with API Responses . . . 25

15 Form Submissions In TypeScript **27**
15.1 Typed Form Events . . . 27
15.2 Form Data Object . . . 27

16 Working with refs & useRef **29**
16.1 Refs with TypeScript . . . 29
16.2 Storing Mutable Values . . . 29
16.3 Refs in Class Components . . . 30

17 Working with "Function Props" **31**
17.1 Callback Props . . . 31
17.2 Generic Callback Props . . . 32

18 Managing State & TypeScript **33**
18.1 useState with Types . . . 33
18.2 useReducer with TypeScript . . . 33
18.3 Custom Hooks with Types . . . 34

19 Adding Styling **36**
19.1 CSS Classes with TypeScript 36
19.2 Inline Styles 36
19.3 Tailwind CSS 37
19.4 CSS-in-JS with Types 37

20 The Context API & TypeScript **38**
20.1 Creating a Typed Context 38
20.2 Using Context 38
20.3 Multiple Context Values 39

21 Summary & Best Practices **41**
21.1 Key Takeaways 41
21.2 Common Mistakes to Avoid 41
21.3 Resources for Learning More 41

Chapter 1

What is TypeScript & Why Use It?

TypeScript is a programming language built on top of JavaScript that adds static type checking. Think of it as JavaScript with extra features that help you catch mistakes before they happen.

1.1 Understanding TypeScript

TypeScript lets you specify what kind of data variables, functions, and components should hold. When you make a mistake—like trying to pass a string where a number is expected—TypeScript catches it immediately during development, not when your code runs.

1.2 Why TypeScript?

- **Catch Errors Early:** Type checking happens during development, not in production
- **Better Code Autocomplete:** Your editor knows what properties and methods are available
- **Self-Documenting Code:** Types serve as documentation for other developers
- **Easier Refactoring:** Changing code is safer because TypeScript tracks all uses
- **Larger Projects:** TypeScript shines when working in teams or large codebases
- **Confidence:** You know your code works as intended before running it

1.3 TypeScript vs JavaScript

JavaScript (No Type Safety):

```
function add(a, b) {
    return a + b;
}
add(5, "10"); // Works but gives "510" not 15!
```

TypeScript (Type Safe):

```
function add(a: number, b: number): number {
    return a + b;
}
add(5, "10"); // Error: Argument of type 'string'
            // is not assignable to type 'number'
```

Chapter 2

Installing & Using TypeScript

2.1 Setting Up TypeScript

Before using TypeScript in your React project, you need to install it:

```
npm install typescript
```

2.2 Creating a TypeScript Configuration

TypeScript needs a configuration file called `tsconfig.json`. Most tools create this automatically:

```
npx tsc --init
```

This creates a configuration file with sensible defaults.

2.3 Compiling TypeScript

TypeScript files (ending in `.ts` or `.tsx`) get compiled to regular JavaScript:

```
npx tsc
```

Your `.ts` files become `.js` files that browsers can understand. Most React tools handle this automatically.

2.4 Using TypeScript in React

For React projects, use Create React App or Vite with TypeScript support:

```
# Create React App
npx create-react-app my-app --template typescript

# Or with Vite
npm create vite@latest my-app -- --template react-ts
```

Chapter 3

Exploring the Base Types

3.1 The Primitive Types

TypeScript has several basic types that cover most data:

```
// string - text data
let name: string = "Alice";

// number - integers and decimals
let age: number = 25;
let price: number = 19.99;

// boolean - true or false
let isStudent: boolean = true;

// null and undefined
let empty: null = null;
let nothing: undefined = undefined;

// any - bypass type checking (avoid when possible!)
let anything: any = "could be anything";
```

3.2 Type Inference

TypeScript can figure out types automatically:

```
let count = 5; // TypeScript knows this is a number
let message = "Hello"; // TypeScript knows this is a string

count = "text"; // Error: cannot assign string to number
```

Chapter 4

Working with Array & Object Types

4.1 Array Types

There are multiple ways to define array types:

```
// Method 1: Type followed by brackets
let numbers: number[] = [1, 2, 3];

// Method 2: Generic Array type
let strings: Array<string> = ["a", "b", "c"];

// Union of types in array
let mixed: (string | number)[] = [1, "two", 3];

// Array of objects
let users: { id: number; name: string }[] = [
    { id: 1, name: "Alice" },
    { id: 2, name: "Bob" }
];
```

4.2 Object Types

Define the structure of objects:

```
// Define object shape
let user: { name: string; age: number } = {
    name: "Alice",
    age: 25
};

// Optional properties with ?
let employee: { id: number; email?: string } = {
    id: 1
    // email is optional, doesn't have to be included
};

// Readonly properties
let config: { readonly apiKey: string } = {
    apiKey: "secret123"
```

```
};
// config.apiKey = "new"; // Error: cannot modify readonly
```

Chapter 5

Understanding Type Inference

5.1 How Type Inference Works

TypeScript automatically deduces types from the values you assign. You don't always need to write types explicitly:

```
// TypeScript infers number
let count = 10;

// TypeScript infers string
let message = "Hello";

// TypeScript infers object structure
let person = { name: "Alice", age: 25 };
// person is inferred as { name: string; age: number }
```

5.2 When Inference Isn't Enough

Some situations require explicit type annotations:

```
// Function parameters need types
function greet(name) { // Error: parameter needs a type
    return "Hello " + name;
}

// Complex structures
let data: { users: { id: number; name: string }[] } = {
    users: [{ id: 1, name: "Alice" }]
};

// Ambiguous values
let value: string | number; // Could be either type
```

Chapter 6

Using Union Types

6.1 Combining Multiple Types

Union types let a value be one of several types:

```
// A variable can be string OR number
let id: string | number;
id = "ABC123"; // Valid
id = 123; // Valid
id = true; // Error: not string or number

// Function that accepts multiple types
function printId(id: string | number) {
    console.log("ID: " + id);
}
printId("ABC");
printId(123);
```

6.2 Type Narrowing

When you use a union type, TypeScript requires you to handle all cases:

```
function printId(id: string | number) {
    // TypeScript doesn't know which type id is
    // console.log(id.toUpperCase()); // Error!

    if (typeof id === "string") {
        console.log(id.toUpperCase()); // OK: string method
    } else {
        console.log(id.toFixed(2)); // OK: number method
    }
}
```

6.3 Real-World Union Example

```
type Status = "pending" | "success" | "error";
```

```
let currentStatus: Status = "pending";
currentStatus = "success"; // Valid
currentStatus = "invalid"; // Error: not a valid status
```

Chapter 7

Understanding Type Aliases

7.1 Creating Reusable Types

Type aliases let you create a name for any type. Use them to avoid repeating complex type definitions:

```
// Define a type once
type User = {
    id: number;
    name: string;
    email: string;
    isActive: boolean;
};

// Use it multiple times
let user1: User = {
    id: 1,
    name: "Alice",
    email: "alice@example.com",
    isActive: true
};

let user2: User = {
    id: 2,
    name: "Bob",
    email: "bob@example.com",
    isActive: false
};
```

7.2 Interfaces vs Type Aliases

Interfaces are similar to type aliases but with some differences:

```
// Type alias - flexible
type PersonType = { name: string; age: number };

// Interface - more structured
interface Person {
    name: string;
```

```
    age: number;
}

// Both work the same for most use cases
let p1: PersonType = { name: "Alice", age: 25 };
let p2: Person = { name: "Bob", age: 30 };

// Key difference: Interfaces can be extended
interface Employee extends Person {
    employeeId: number;
}
```

7.3 Type Aliases with Unions

Type aliases are great for union types (interfaces cannot do this):

```
type ID = string | number;
type Status = "pending" | "complete" | "failed";
type Response = string | null | undefined;
```

Chapter 8

Functions & Function Types

8.1 Basic Function Types

Annotate function parameters and return types:

```
// Function with typed parameters and return type
function add(a: number, b: number): number {
    return a + b;
}

// Function with multiple parameters
function greet(firstName: string, lastName: string): string {
    return `Hello ${firstName} ${lastName}`;
}

// Function that returns nothing (void)
function logMessage(message: string): void {
    console.log(message);
}

// Optional parameters with ?
function describe(name: string, age?: number): string {
    return age ? `${name} is ${age}` : name;
}
```

8.2 Function Type Expressions

Define the structure of functions as types:

```
// Define a function type
type MathOperation = (a: number, b: number) => number;

// Use the type
const add: MathOperation = (a, b) => a + b;
const subtract: MathOperation = (a, b) => a - b;

// Function that accepts a function as parameter
function performOperation(
    x: number,
```

```
    y: number,
    operation: MathOperation
): number {
    return operation(x, y);
}

performOperation(5, 3, add); // Returns 8
```

8.3 Callback Functions

Functions that accept other functions as parameters:

```
function processItems(
    items: string[],
    callback: (item: string) => void
): void {
    items.forEach(item => {
        callback(item);
    });
}

processItems(["a", "b", "c"], (item) => {
    console.log(item.toUpperCase());
});
```

Chapter 9

Diving Into Generics

9.1 What Are Generics?

Generics let you write code that works with multiple types while still maintaining type safety. Think of them as type placeholders:

```
// Without generics - loses type information
function getFirst(arr: any[]): any {
    return arr[0]; // Returns 'any', we lose type info
}

// With generics - preserves type information
function getFirstGeneric<T>(arr: T[]): T {
    return arr[0]; // Returns type T
}

const firstNum = getFirstGeneric([1, 2, 3]); // T is number
const firstStr = getFirstGeneric(["a", "b"]); // T is string
```

9.2 Generic Functions

```
// Generic function that works with any type
function printArray<T>(items: T[]): void {
    items.forEach(item => console.log(item));
}

// Multiple type parameters
function combine<T, U>(a: T, b: U): (T | U)[] {
    return [a, b];
}

combine(1, "two"); // [1, "two"]
combine("hello", true); // ["hello", true]

// Generic with constraints
function getLength<T extends { length: number }>(x: T): number {
    return x.length; // T must have a length property
}
```

```
getLength("hello"); // 5
getLength([1, 2, 3]); // 3
getLength({ length: 10 }); // 10
```

Chapter 10

A Closer Look At Generics

10.1 Generic Interfaces

```
// Generic interface
interface Container<T> {
    value: T;
    getValue(): T;
    setValue(value: T): void;
}

// Implement with specific type
const numberContainer: Container<number> = {
    value: 42,
    getValue() { return this.value; },
    setValue(value) { this.value = value; }
};

const stringContainer: Container<string> = {
    value: "hello",
    getValue() { return this.value; },
    setValue(value) { this.value = value; }
};
```

10.2 Generic Constraints

Limit what types can be used with generics:

```
// T must be a string or number
function process<T extends string | number>(value: T): T {
    return value;
}

// T must be an object with specific properties
interface HasName {
    name: string;
}

function getName<T extends HasName>(obj: T): string {
```

```
    return obj.name;
}

// Extends another type parameter
function merge<T, U extends T>(obj1: T, obj2: U): T & U {
    return { ...obj1, ...obj2 };
}
```

10.3 Default Type Parameters

```
// Generic with default type
interface Box<T = string> {
    content: T;
}

const stringBox: Box = { content: "hello" }; // T defaults to string
const numberBox: Box<number> = { content: 42 }; // T is number

// Function with default type
function create<T = string>(value: T): T {
    return value;
}

create(); // Uses default string type
```

Chapter 11

Creating a React + TypeScript Project

11.1 Setup with Create React App

```
npx create-react-app my-app --template typescript
cd my-app
npm start
```

This creates a new React project with TypeScript support.

11.2 Setup with Vite

```
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install
npm run dev
```

11.3 Project Structure

Your new project includes:

- `src/App.tsx` - Main React component
- `src/main.tsx` - Entry point
- `tsconfig.json` - TypeScript configuration
- `package.json` - Dependencies

11.4 First TypeScript React File

```tsx
// src/App.tsx
import React from 'react';

const App: React.FC = () => {
    return <h1>Hello TypeScript + React!</h1>;
};

export default App;
```

Chapter 12

Working with Components & TypeScript

12.1 Functional Components

Define React functional components with types:

```
import React from 'react';

// Method 1: React.FC (Function Component)
const Welcome: React.FC = () => {
    return <div>Welcome!</div>;
};

// Method 2: Plain function with return type
function Goodbye(): React.ReactElement {
    return <div>Goodbye!</div>;
}

// Method 3: Arrow function with type annotation
const Hello = (): JSX.Element => {
    return <div>Hello!</div>;
};
```

12.2 Class Components

Type-safe class components:

```
import React from 'react';

interface State {
    count: number;
}

class Counter extends React.Component<{}, State> {
    constructor(props: {}) {
        super(props);
        this.state = { count: 0 };
```

```
    }

    render() {
        return (
            <div>
                <p>Count: {this.state.count}</p>
                <button onClick={() =>
                    this.setState({ count: this.state.count + 1 })
                }>
                    Increment
                </button>
            </div>
        );
    }
}

export default Counter;
```

Chapter 13

Working with Props & TypeScript

13.1 Typing Props

Define the shape of component props:

```
interface ButtonProps {
    label: string;
    onClick: () => void;
    disabled?: boolean;
    variant?: "primary" | "secondary";
}

const Button: React.FC<ButtonProps> = ({
    label,
    onClick,
    disabled = false,
    variant = "primary"
}) => {
    return (
        <button
            onClick={onClick}
            disabled={disabled}
            className={variant}
        >
            {label}
        </button>
    );
};

// Using the component
<Button
    label="Click me"
    onClick={() => console.log("clicked")}
    variant="primary"
/>
```

13.2 Common Prop Types

```
interface FormProps {
    // Functions
    onSubmit: (data: string) => void;
    onChange: (value: string) => void;

    // Elements
    children: React.ReactNode;

    // Multiple possible values
    size: "small" | "medium" | "large";

    // Optional values
    title?: string;

    // Arrays
    items: Array<{ id: number; label: string }>;
}
```

13.3 Spread Props

Safely spread props with types:

```
interface InputProps
    extends React.InputHTMLAttributes<HTMLInputElement> {
    label: string;
}

const TextInput: React.FC<InputProps> = ({
    label,
    ...inputProps
}) => {
    return (
        <div>
            <label>{label}</label>
            <input {...inputProps} />
        </div>
    );
};

// Can use all standard input properties
<TextInput
    label="Email"
    type="email"
    placeholder="Enter email"
    required
/>
```

Chapter 14

Adding a Data Model

14.1 Creating Type Definitions

Define data structures for your application:

```
// types/models.ts
export interface User {
    id: number;
    name: string;
    email: string;
    createdAt: Date;
}

export interface Product {
    id: number;
    title: string;
    price: number;
    description: string;
    inStock: boolean;
}

export interface Order {
    id: number;
    userId: number;
    products: Product[];
    total: number;
    status: "pending" | "shipped" | "delivered";
}
```

14.2 Working with API Responses

```
import { User, Product } from './types/models';

interface ApiResponse<T> {
    success: boolean;
    data: T;
    message?: string;
}
```

```
// Fetching users
async function getUsers(): Promise<ApiResponse<User[]>> {
    const response = await fetch('/api/users');
    return response.json();
}

// Fetching a single product
async function getProduct(id: number): Promise<Product> {
    const response = await fetch(`/api/products/${id}`);
    const data = await response.json();
    return data;
}
```

Chapter 15

Form Submissions In TypeScript

15.1 Typed Form Events

```
import React, { useState } from 'react';

const LoginForm: React.FC = () => {
   const [email, setEmail] = useState<string>("");
   const [password, setPassword] = useState<string>("");

   const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
      e.preventDefault();
      console.log("Email:", email);
      console.log("Password:", password);
   };

   return (
      <form onSubmit={handleSubmit}>
         <input
            type="email"
            value={email}
            onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
               setEmail(e.target.value)
            }
         />
         <input
            type="password"
            value={password}
            onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
               setPassword(e.target.value)
            }
         />
         <button type="submit">Login</button>
      </form>
   );
};
```

15.2 Form Data Object

```tsx
interface FormData {
    firstName: string;
    lastName: string;
    email: string;
    age: number;
}

const UserForm: React.FC = () => {
    const [formData, setFormData] = useState<FormData>({
        firstName: "",
        lastName: "",
        email: "",
        age: 0
    });

    const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
        const { name, value } = e.target;
        setFormData(prev => ({
            ...prev,
            [name]: name === "age" ? parseInt(value) : value
        }));
    };

    const handleSubmit = (e: React.FormEvent) => {
        e.preventDefault();
        console.log("Form submitted:", formData);
    };

    return (
        <form onSubmit={handleSubmit}>
            <input
                name="firstName"
                value={formData.firstName}
                onChange={handleChange}
            />
            {/* More inputs */}
        </form>
    );
};
```

Chapter 16

Working with refs & useRef

16.1 Refs with TypeScript

Access DOM elements directly:

```
import React, { useRef } from 'react';

const TextInput: React.FC = () => {
    const inputRef = useRef<HTMLInputElement>(null);

    const focusInput = () => {
        inputRef.current?.focus();
    };

    const clearInput = () => {
        if (inputRef.current) {
            inputRef.current.value = "";
        }
    };

    return (
        <>
            <input ref={inputRef} type="text" />
            <button onClick={focusInput}>Focus</button>
            <button onClick={clearInput}>Clear</button>
        </>
    );
};
```

16.2 Storing Mutable Values

```
import React, { useRef, useEffect } from 'react';

const Timer: React.FC = () => {
    const intervalRef = useRef<NodeJS.Timeout | null>(null);
    const countRef = useRef<number>(0);

    const startTimer = () => {
```

```
        intervalRef.current = setInterval(() => {
            countRef.current++;
            console.log("Time:", countRef.current);
        }, 1000);
    };

    const stopTimer = () => {
        if (intervalRef.current) {
            clearInterval(intervalRef.current);
        }
    };

    return (
        <>
            <button onClick={startTimer}>Start</button>
            <button onClick={stopTimer}>Stop</button>
        </>
    );
};
```

16.3 Refs in Class Components

```
import React from 'react';

class VideoPlayer extends React.Component {
    private videoRef = React.createRef<HTMLVideoElement>();

    play = () => {
        this.videoRef.current?.play();
    };

    pause = () => {
        this.videoRef.current?.pause();
    };

    render() {
        return (
            <>
                <video ref={this.videoRef} />
                <button onClick={this.play}>Play</button>
                <button onClick={this.pause}>Pause</button>
            </>
        );
    }
}
```

Chapter 17

Working with "Function Props"

17.1 Callback Props

Pass functions as props to child components:

```
interface ListItemProps {
    item: string;
    onDelete: (id: string) => void;
    onEdit: (id: string, newValue: string) => void;
}

const ListItem: React.FC<ListItemProps> = ({
    item,
    onDelete,
    onEdit
}) => {
    return (
        <div>
            <p>{item}</p>
            <button onClick={() => onDelete(item)}>Delete</button>
            <button onClick={() => onEdit(item, "updated")}>Edit</button>
        </div>
    );
};

// Parent component
const TodoList: React.FC = () => {
    const handleDelete = (id: string) => {
        console.log("Delete:", id);
    };

    const handleEdit = (id: string, newValue: string) => {
        console.log("Edit:", id, newValue);
    };

    return (
        <ListItem
            item="Buy groceries"
            onDelete={handleDelete}
            onEdit={handleEdit}
```

```
        />
    );
};
```

17.2 Generic Callback Props

```
interface SelectProps<T> {
    options: T[];
    onSelect: (value: T) => void;
    getLabel: (item: T) => string;
}

const Select = <T,>({
    options,
    onSelect,
    getLabel
}: SelectProps<T>) => {
    return (
        <select onChange={(e) => {
            const selected = options[parseInt(e.target.value)];
            onSelect(selected);
        }}>
            {options.map((option, i) => (
                <option key={i} value={i}>
                    {getLabel(option)}
                </option>
            ))}
        </select>
    );
};

// Usage
interface User {
    id: number;
    name: string;
}

<Select<User>
    options={users}
    onSelect={(user) => console.log(user.name)}
    getLabel={(user) => user.name}
/>
```

Chapter 18

Managing State & TypeScript

18.1 useState with Types

```
import React, { useState } from 'react';

// Type inference
const [count, setCount] = useState(0); // number
const [name, setName] = useState(""); // string

// Explicit types
const [isLoading, setIsLoading] = useState<boolean>(false);
const [error, setError] = useState<string | null>(null);

// Complex state
interface User {
    id: number;
    name: string;
    email: string;
}

const [user, setUser] = useState<User | null>(null);
const [users, setUsers] = useState<User[]>([]);

// Union types for state
type Status = "idle" | "loading" | "success" | "error";
const [status, setStatus] = useState<Status>("idle");
```

18.2 useReducer with TypeScript

```
import React, { useReducer } from 'react';

interface State {
    count: number;
    message: string;
}

type Action =
```

```
    | { type: "INCREMENT" }
    | { type: "DECREMENT" }
    | { type: "SET_MESSAGE"; payload: string }
    | { type: "RESET" };

const reducer = (state: State, action: Action): State => {
    switch(action.type) {
        case "INCREMENT":
            return { ...state, count: state.count + 1 };
        case "DECREMENT":
            return { ...state, count: state.count - 1 };
        case "SET_MESSAGE":
            return { ...state, message: action.payload };
        case "RESET":
            return { count: 0, message: "" };
        default:
            return state;
    }
};

const Counter: React.FC = () => {
    const [state, dispatch] = useReducer(reducer, {
        count: 0,
        message: ""
    });

    return (
        <>
            <p>Count: {state.count}</p>
            <p>Message: {state.message}</p>
            <button onClick={() => dispatch({ type: "INCREMENT" })}>
                Increment
            </button>
        </>
    );
};
```

18.3 Custom Hooks with Types

```
import { useState } from 'react';

interface UseBooleanReturn {
    value: boolean;
    toggle: () => void;
    setTrue: () => void;
    setFalse: () => void;
}

function useBoolean(initialValue: boolean = false): UseBooleanReturn {
    const [value, setValue] = useState(initialValue);

    return {
```

```
        value,
        toggle: () => setValue(!value),
        setTrue: () => setValue(true),
        setFalse: () => setValue(false)
    };
}

// Usage
const { value, toggle } = useBoolean(false);
```

Chapter 19

Adding Styling

19.1 CSS Classes with TypeScript

```
import React from 'react';
import './Button.css';

interface ButtonProps {
    variant: "primary" | "secondary" | "danger";
    size: "small" | "medium" | "large";
    label: string;
}

const Button: React.FC<ButtonProps> = ({
    variant,
    size,
    label
}) => {
    const className = `btn btn--${variant} btn--${size}`;
    return <button className={className}>{label}</button>;
};
```

19.2 Inline Styles

```
import React from 'react';

interface BoxProps {
    backgroundColor: string;
    padding: number;
}

const Box: React.FC<BoxProps> = ({ backgroundColor, padding }) => {
    const styles: React.CSSProperties = {
        backgroundColor,
        padding: `${padding}px`,
        borderRadius: "8px"
    };
```

```
    return <div style={styles}>Styled Box</div>;
};

// Usage
<Box backgroundColor="lightblue" padding={20} />
```

19.3 Tailwind CSS

```
interface CardProps {
    title: string;
    highlighted?: boolean;
}

const Card: React.FC<CardProps> = ({ title, highlighted }) => {
    const bgClass = highlighted ? "bg-blue-100" : "bg-white";

    return (
        <div className={`${bgClass} p-4 rounded-lg shadow-md`}>
            <h3 className="text-lg font-bold">{title}</h3>
        </div>
    );
};
```

19.4 CSS-in-JS with Types

```
import styled from 'styled-components';

interface StyledButtonProps {
    isActive: boolean;
}

const StyledButton = styled.button<StyledButtonProps>`
    background-color: ${props => props.isActive ? "blue" : "gray"};
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 4px;
    cursor: pointer;

    &:hover {
        opacity: 0.9;
    }
`;

// Usage
<StyledButton isActive={true}>Click me</StyledButton>
```

Chapter 20

The Context API & TypeScript

20.1 Creating a Typed Context

```
import React, { createContext, useContext } from 'react';

interface ThemeContextType {
    theme: "light" | "dark";
    toggleTheme: () => void;
}

const ThemeContext = createContext<ThemeContextType | undefined>(undefined);

// Provider component
export const ThemeProvider: React.FC<{ children: React.ReactNode }> = ({
    children
}) => {
    const [theme, setTheme] = React.useState<"light" | "dark">("light");

    const toggleTheme = () => {
        setTheme(theme === "light" ? "dark" : "light");
    };

    return (
        <ThemeContext.Provider value={{ theme, toggleTheme }}>
            {children}
        </ThemeContext.Provider>
    );
};
```

20.2 Using Context

```
// Custom hook for using context
function useTheme(): ThemeContextType {
    const context = useContext(ThemeContext);
    if (context === undefined) {
        throw new Error("useTheme must be used within ThemeProvider");
    }
```

```
    return context;
}

// Component using context
const ThemedButton: React.FC = () => {
    const { theme, toggleTheme } = useTheme();

    return (
        <button onClick={toggleTheme}>
            Current theme: {theme}
        </button>
    );
};

// App setup
const App: React.FC = () => {
    return (
        <ThemeProvider>
            <ThemedButton />
        </ThemeProvider>
    );
};
```

20.3 Multiple Context Values

```
interface User {
    id: number;
    name: string;
}

interface AuthContextType {
    user: User | null;
    isAuthenticated: boolean;
    login: (user: User) => void;
    logout: () => void;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export const AuthProvider: React.FC<{ children: React.ReactNode }> = ({
    children
}) => {
    const [user, setUser] = React.useState<User | null>(null);

    const login = (newUser: User) => setUser(newUser);
    const logout = () => setUser(null);

    return (
        <AuthContext.Provider
            value={{
                user,
                isAuthenticated: user !== null,
```

```
            login,
            logout
        }}
    >
        {children}
    </AuthContext.Provider>
  );
};

// Custom hook
function useAuth(): AuthContextType {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error("useAuth must be used within AuthProvider");
  }
  return context;
}
```

Chapter 21

Summary & Best Practices

21.1 Key Takeaways

- **Always type your props** - Makes components reusable and safe
- **Use interfaces for complex shapes** - Better organization than inline types
- **Leverage type inference** - Don't type everything explicitly
- **Generics for reusable logic** - Write once, use with multiple types
- **Strict mode is your friend** - Enable it for better type safety
- **Extract types to separate files** - Keep your code clean and organized
- **Use React.FC for clarity** - Explicitly mark React components

21.2 Common Mistakes to Avoid

- **Using `any`** - Defeats the purpose of TypeScript
- **Over-complicating types** - Keep them simple and readable
- **Ignoring type errors** - Address them, don't suppress them
- **Forgetting null checks** - Use optional chaining (?.) and nullish coalescing (??)
- **Not using the Context custom hook** - Always throw an error if context is undefined
- **Typing function parameters as `any`** - Always specify proper types

21.3 Resources for Learning More

- **TypeScript Handbook:** The official TypeScript documentation
- **React TypeScript Cheatsheet:** Community-maintained guide for React + TypeScript
- **Visual Studio Code:** Best IDE for TypeScript with excellent type hints
- **Type Safety Communities:** Join TypeScript communities online for questions